

Onboarding → Clubify — Sync API (contrato de integración)

Estado: Fases A (diccionario) + B (token) + C (endpoints) + D (**webhook business.activated**) — **TODAS LIVE**. El onboarding se adapta a estos nombres/endpoints; Clubify NO se reestructura. Fuente de verdad: `backend/prisma/schema.prisma`. Nombres y tipos copiados **tal cual** existen hoy.

Modelo mental (leer primero)

- **Un negocio = una fila `Tenant`**. Su id único es `Tenant.id` (`uuid`) y su identificador público de URL es `Tenant.slug` (`@unique`).
- Casi toda fila de un negocio lleva un escalar `tenantId` (FK a `Tenant`, `onDelete: Cascade`): `Card.tenantId`, `Product.tenantId`, `Category.tenantId`, `Location.tenantId`, `Promotion.tenantId`, etc. **Ese `tenantId` es la clave de aislamiento** que la Sync API fija en cada escritura.
- Encima del negocio está la marca: `Tenant.whiteLabelId` → `WhiteLabel` (negocio → marca). La config de marca (branding fallback, módulos) vive en `WhiteLabel` / `WhiteLabelModule`, **no** por negocio.
- `Setting` es un key-value **global** (no tenant-scoped por columna). La config del negocio NO vive ahí.

FASE A — Diccionario de campos (entregable #1)

1. Negocio / cuenta

Concepto	Modelo	Campo (exacto)	Tipo	Notas
Nombre interno	Tenant	name	String	
Nombre de marca (público)	Tenant	brandName	String	Se muestra al cliente / en el wallet
Slug (id público)	Tenant	slug	String @unique	Identificador URL
Categoría	Tenant	businessCategorySlug	String?	Slug de <code>common/business-categories.ts</code> (<code>restaurant</code> , <code>barbershop</code> ...). No es FK. Fallback runtime <code>'restaurant'</code> .
Descripción	Storefront	description	String @default("")	Tenant NO tiene descripción. Vive en el storefront.
Dirección	Location	address	String @default("")	Por sede, no en Tenant. Un negocio puede tener varias <code>Location</code> .
Ciudad	—	—	—	No hay <code>city</code> de primera clase. Cercanos: <code>Tenant.trialCity</code> (captura de prospecto), <code>Location.state</code> .
País	Tenant	country	String @default("C0")	ISO 3166-1 alpha-2
Teléfono del negocio	Tenant	phone	String?	Este es el teléfono. Distinto de los <code>whatsapp*</code> .
WhatsApp general	Tenant	whatsappPhone	String?	
WhatsApp pedidos	Tenant	whatsappOrdersPhone	String?	Cae a <code>whatsappPhone</code>
WhatsApp domicilios	Tenant	whatsappDeliveryPhone	String?	
WhatsApp reservas	Tenant	whatsappReservationsPhone	String?	
Correo	Tenant	email	String	Requerido
Sitio web	—	—	—	No existe campo <code>website</code> en Tenant.

2. Branding

Concepto	Modelo	Campo	Tipo	Notas
Logo	Tenant	logoUrl	String?	
Logo wallet	Tenant	walletLogoUrl	String?	Cae a logoUrl
Logo push	Tenant	pushLogoUrl	String?	Cae a walletLogoUrl → logoUrl
Color primario	Tenant	primaryColor	String @default("#22C55E")	
Color secundario	Tenant	secondaryColor	String @default("#15803D")	
Imagen de portada	Storefront	heroImageUrl	String?	Tenant no tiene portada
Colores de texto (storefront)	Storefront	titleColor , descriptionColor , logoBgColor	String?	
Color de botones	—	Storefront.theme (Json) / por-popup	—	No hay "color de botón" global en Tenant
Favicon / ícono	WhiteLabel	faviconUrl , iconUrl	String?	Favicon es a nivel MARCA, no por negocio.

La **tarjeta de sellos** tiene su propio branding (ver §3): Card.primaryColor , Card.logoUrl , Card.heroImageUrl , Card.iconUrl , Card.stampIcon .

3. Programa de sellos / lealtad → model Card (NO existe LoyaltyProgram)

Card.type ∈ CardType { STAMPS, POINTS, DISCOUNT, MEMBERSHIP, COUPON, GIFT, MULTI, CASHBACK, VISITS, HYBRID } . Varias por tenant.

Concepto	Campo	Tipo
Nombre	name	String
Sellos requeridos	stampsRequired	Int?
Premio	rewardText	String @default("")
Descripción del premio	rewardDescText	String @default("")
Descripción	description	String @default("")
Mensaje al completar	rewardEarnedMessage	String @default("")
Mensaje al sellar	stampEarnedMessage	String @default("") ([#] =restantes)
Color tarjeta	primaryColor / secondaryColor	String
Ícono del sello (emoji)	stampIcon	String @default("👉")
Logo / portada / ícono	logoUrl? , heroImageUrl? , iconUrl?	String?
Términos	terms / termsEnabled	String / Boolean
Monto mínimo por sello	minAmountPerStamp	Decimal?
Vigencia	validFrom? , validUntil? , validDaysAfterIssue?	Date? / Int?
Activa	isActive	Boolean @default(true)

Instancia por cliente = Pass (stampsCount , status PassStatus{ACTIVE,COMPLETED,REVOKED}). Evento de sello = Stamp .

4. Cupones → también model Card (type = COUPON | DISCOUNT | GIFT)

No hay modelo Coupon . Sí soporta múltiples por negocio (Tenant 1→N Card).

Concepto	Campo en Card	Notas
Nombre	name	
Descripción	description	
Imagen	heroImageUrl / logoUrl / iconUrl	
Código	—	Card NO tiene campo code . Se redime por QR del pase (Pass.qrToken).
Cantidad disponible	—	No hay availableQuantity en Card (sí en Promotion.maxRedemptions).
Fecha inicio/fin	validFrom / validUntil (o validDaysAfterIssue)	
Condiciones	terms	Texto libre

model Promotion es el mejor match si "cupón" significa descuento con cantidad/condiciones: type PromotionType{DISCOUNT_PCT, DISCOUNT_AMOUNT, BUY_X_GET_Y, COMBO, FREE_ITEM}, value Decimal, originalPrice Decimal?, conditions Json, maxRedemptions Int?, maxRedemptionsPerCustomer Int?, validFrom/validUntil, isActive . **Tampoco tiene code** . El descuento se aplica al carrito solo si conditions.productIds está seteado (ver fix PDF454).

5. Menú digital → Category → Product (+ ProductVariant, ProductExtra, Adicional)

Modelo	Campo	Notas
Category	tenantId, parentId? (subcats), name, slug (@@unique[tenantId,parentId,slug]), description?, imageUrl?, position, isActive	
Product	tenantId, categoryId? (FK onDelete: SetNull), name, description, basePrice Decimal(10,2), imageUrl?, isAvailable Boolean, isRecommended Boolean (destacado), availableForMesa/Delivery, position, stock?	Precio = basePrice
ProductVariant	productId, groupName, name, priceDelta, isDefault, position	
ProductExtra	productId, name, price, maxQty, isAvailable	

Cadena: Tenant 1→N Category 1→N Product 1→N Variant/Extra .

6. Horarios → model ServiceAvailability (solo módulo de citas)

No hay modelo de "horario de apertura" general del storefront. Una fila por intervalo abierto.

Campo	Tipo	Notas
tenantId	String	
providerId	String?	null = a nivel negocio
weekday	Int	0=domingo ... 6=sábado
startMin / endMin	Int	Minutos desde medianoche (540=09:00) en TZ del tenant

Día sin filas = cerrado. Overrides por fecha = ServiceException (date @db.Date, closed, startMin?, endMin?, @@unique[tenantId,date]).

Slots de reserva de MESA = Tenant.reservationSlots String[] (array "HH:MM"), distinto.

7. Google Reviews

Modelo	Campo	Notas
Tenant	googleReviewUrl (String?)	URL de "pide más reseñas". /r/{slug} : 4-5★→Google, <4★→feedback privado

Relacionados: Tenant.mapsUrl , Location.mapsUrl .

8. Redes / contacto

Concepto	Modelo.Campo	Notas
WhatsApp	Tenant.whatsappPhone (+ whatsappOrders/Delivery/Reservations Phone)	ver §1
Instagram	Tenant.instagramUrl	
Facebook	Tenant.facebookUrl	
TikTok	—	No existe campo TikTok.
Web	—	No existe campo web.
Correo	Tenant.email	
Maps	Tenant.mapsUrl	

9. Módulos / funcionalidades activables (4 capas)

(A) **Flags booleanos en Tenant** : reservationsEnabled (mesa), serviceReservationsEnabled (citas), reviewAlertsEnabled , billingAlertsEnabled (default true), deliveryAlertsEnabled , whatsappFeedbackEnabled , tutorialsEnabled , academyEnabled .
(B) **Flags en Storefront** : ordersEnabled , ordersDeliveryEnabled , digitalMenuEnabled , bookMenuEnabled , whatsappButtonEnabled , isPublished . (C) **Marca → WhiteLabelModule + enum ModuleKey** : valores exactos **REFERRALS, ORDERS, GROW_BUSINESS_SMS, REVIEWS, SERVICE_RESERVATIONS, COMMUNITY** . getMine deriva reviewsEnabled / referralsEnabled / communityEnabled . (D) **Categoría implica secciones**: BusinessCategory.modules[] (cards|customers|scanner|push|menu|orders|analytics|staff|info_links|services), estático por businessCategorySlug .

No hay un booleano único de wallet/geopush/promos/beneficios: wallet siempre on (Pass), geopush por Location (lat/lng/radius), promos = existencia de Promotion , beneficios = subsistema Cuponera (BenefitCampaign) .

10. Identificador único + activación

- business_id** = Tenant.id (uuid); identificador público = Tenant.slug .
- Estado de activación** = Tenant.status → TenantStatus { ACTIVE, SUSPENDED, TRIAL } (@default(TRIAL)) .
- Otros: isLocked/lockedAt/lockedReason (lock demo), deletedAt (soft-delete), suspendedAt , trialStartedAt/trialEndsAt/gracePeriodDays , currentPeriodEnd , gateway (stripeCustomerId , hotmartSubscriberCode), isCampaignHost (tenant "de sistema").
- Publicación del menú** (aparte del status): Storefront.isPublished .
- Dónde se togglea**: admin/super-admin vía tenants.controller (UpdateTenantDto : status , reservationsEnabled , serviceReservationsEnabled ...); dueño vía PATCH /tenants/me (UpdateMyBody) — el dueño **no** cambia status / slug / name .

Gotchas clave para el sync

- Teléfono del negocio** = Tenant.phone (≠ whatsappPhone) .
- Dirección/ciudad NO están en Tenant** (dirección = Location.address ; no hay city / website) .
- Cupones y sellos** = el MISMO modelo Card (discriminado por type); sin campo code .
- Horarios** = solo ServiceAvailability (minutos-desde-medianoche, weekday 0=domingo).
- Módulos repartidos** en Tenant + Storefront + WhiteLabelModule + categoría.

FASE B — Token por negocio (plan, sin construir)

Objetivo: que sea imposible escribir en el negocio equivocado. Nunca una API key global.

Schema (migración nueva):

```

model OnboardingToken {
  id      String    @id @default(uuid())
  tenantId String      // ← el token SÓLO puede tocar este negocio
  tokenHash String    @unique      // sha-256 del token; el claro se muestra 1 sola vez
  label   String    @default("Onboarding")
  createdAt DateTime @default(now())
  lastUsedAt DateTime?
  revokedAt DateTime?
  tenant   Tenant    @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  @@index([tenantId])
}

```

- **Generación:** en la config del negocio, botón **"Conectar con Onboarding"** → `POST /tenants/me/onboarding-token` genera 32 bytes random, guarda `sha256(token)`, devuelve el token en claro **una vez** + el `business_id` (`tenant.id`).
- **Revocar:** `DELETE /tenants/me/onboarding-token/:id` (set `revokedAt`).
- **Autenticación:** header `Authorization: Bearer <token>`. Un `OnboardingAuthGuard` hace `sha256(token)`, busca `OnboardingToken` con `revokedAt=null`, resuelve `tenantId` y lo inyecta en el request. **Todo endpoint de sync opera SOLO sobre ese `tenantId`** (nunca acepta `tenantId` del body).

FASE C — Endpoints de sincronización (plan)

LIVE. Base `https://api.soyclubify.com/api/sync/*`. **Auth:** header `Authorization: Bearer <token>` (el token de Fase B). El `tenantId` SIEMPRE sale del token — ningún endpoint acepta un id de negocio en el body. Content-Type `application/json`.

Semántica: upsert **NO destructivo** (crea o actualiza, nunca borra lo que el negocio agregó a mano), EXCEPTO `/hours` que reemplaza el set completo de horarios a nivel negocio. Solo se tocan los campos presentes en el body (omitir un campo = no cambiarlo; enviarlo en `null / ""` = limpiarlo, salvo los no-nulos como `name/color` que se ignoran si van vacíos). **Idempotencia** por llaves estables: categorías por `slug(name)`, productos por `(name, categoría)`, cupones por `name`. (Si el onboarding renombra, se crea una fila nueva — mantener nombres estables o pedir `externalId` en una iteración futura.)

Endpoint	Body (campos → destino real)	Respuesta
PATCH /sync/business	name, brandName, email (se ignoran si vacíos) · businessCategorySlug (se slugifica) · country (2 letras) · phone, whatsappPhone, whatsappOrdersPhone, whatsappDeliveryPhone, whatsappReservationsPhone → Tenant.*	{ok, updated:[...campos]}
PATCH /sync/branding	logoUrl, walletLogoUrl, pushLogoUrl, primaryColor, secondaryColor → Tenant.* · heroImageUrl → Storefront.heroImageUrl (upsert)	{ok}
PATCH /sync/contact	instagramUrl, facebookUrl, mapsUrl, whatsappPhone → Tenant.*	{ok, updated}
PATCH /sync/reviews	googleReviewUrl → Tenant.googleReviewUrl	{ok}
PUT /sync/location	address, mapsUrl, latitude, longitude, name → la primera Location del negocio (crea si no hay; lat/lng default 0)	{ok, location_id, created}
PUT /sync/loyalty-card	name (req. al crear), stampsRequired, rewardText, rewardDescText, description, rewardEarnedMessage, stampEarnedMessage, stampIcon, primaryColor, secondaryColor → la Card type=STAMPS del negocio (crea si no hay)	{ok, card_id, created}
PUT /sync/hours	arreglo (o {items:[...]}) de {weekday(0=dom... 6=sáb), startMin, endMin} (minutos-desde-medianoche) → reemplaza las ServiceAvailability a nivel negocio (providerId null)	{ok, count}
PATCH /sync/modules	digitalMenu, orders, ordersDelivery, published → Storefront.*Enabled / isPublished · reservations, serviceReservations → Tenant.*Enabled (booleans)	{ok}
POST /sync/categories	arreglo (o {items}) de {name*, description, imageUrl, position} → Category (upsert por slug, parent nivel raíz)	{ok, categories:[{name,slug,id,created}]}
POST /sync/products	arreglo de {name*, basePrice* (al crear), description, imageUrl, isAvailable, isRecommended, position, categorySlug categoryName} → Product (upsert por name+categoria; resuelve categoria por slug)	{ok, products:[{name,id,created}]}
POST /sync/coupons	arreglo de {name*, description, terms, imageUrl(→heroImageUrl), validFrom, validUntil (ISO)} → Card type=COUPON (upsert por name)	{ok, coupons:[{name,id,created}]}
POST /sync/activate	(sin body) → Tenant.status=ACTIVE + Storefront.isPublished=true · Fase D disparará el webhook aquí	{ok, business_id, name, phone, slug, status}
GET /sync/whoami	(sin body) verifica el token	{business_id, name, brandName, slug, status}

* = requerido al **crear**. Errores → 400 con {message} ; token inválido/ausente → 401 .

Ejemplo:

```
curl -X POST https://api.soyclubify.com/api/sync/products \
-H "Authorization: Bearer clbf_xxxxx" -H "Content-Type: application/json" \
-d ' [{ "name": "Latte", "basePrice": 12000, "categoryName": "Bebidas", "isRecommended": true } ]'
```

FASE D — Webhook de activación (LIVE)

Cuando un negocio pasa a `ACTIVE` — vía `POST /sync/activate` o al **activarlo desde el panel/simulador** (Master Admin) — Clubify hace un `POST` firmado a una URL configurable:

```
{
  "event": "business.activated",
  "business_id": "<Tenant.id>",
  "name": "<brandName>",
  "phone": "<Tenant.phone>",
  "slug": "<Tenant.slug>",
  "activated_at": "<ISO>",
  "sent_at": "<ISO>"
}
```

Headers: `Content-Type: application/json` · `X-Clubify-Event: business.activated` · `X-Clubify-Signature: sha256=<hmac_sha256(secret, rawBody)>`.

Verificar la firma (ejemplo Node): `crypto.createHmac('sha256', SECRET).update(rawBody).digest('hex')` y comparar con el valor tras `sha256=`.

Configuración (Master Admin → `/superadmin/integraciones` → "Webhook de Onboarding"): URL destino + secreto de firma + toggle "Activo" + botón **Probar** (envía un `webhook.test`). También por API (`@Roles PLATFORM_OWNER`): `GET/PUT /onboarding-webhook`, `POST /onboarding-webhook/test`. Config global en `Setting` (`onboarding.webhook.url|secret|enabled`).

- **Best-effort:** el envío es fire-and-forget con timeout de 6 s; si falla, **nunca** rompe ni retrasa la activación (solo queda un warning en logs).
- Clubify solo **emite**; el onboarding reacciona con su propia mensajería.
- Si el toggle está apagado o no hay URL, no se envía nada.

Estado del build: A (diccionario)  · B (token+guard)  · C (endpoints por entidad)  · D (webhook `business.activated`)  — **TODO LIVE**. Se dispara desde `POST /sync/activate` y desde la activación por panel/simulador; se configura en `/superadmin/integraciones`.